
Why Agent Skill Is the Best Implementation Mechanism for the “Action Plan” within the KSTAR Learning Loop, and Why Hermes Agent Resembles the NEOLAF Agent

Zhang Hao

Independent reading note prepared for instructor review
Based on the NEOLAF paper and the local Hermes Agent implementation

Abstract

This note examines two tightly related questions raised by the NEOLAF framework. First, why is agent skill the most suitable implementation mechanism for the “action plan” inside the KSTAR learning loop? Second, why does Hermes Agent look structurally similar to the NEOLAF agent? The central argument is that Action in KSTAR cannot remain a transient piece of generated advice. It must become a structured, executable, reusable, interpretable, and updatable unit of procedural knowledge. A skill satisfies exactly these requirements: it packages what to do, under what conditions, with which tools, in what order, and how success should be verified. Moreover, Hermes Agent explicitly treats skills as procedural memory and combines them with persistent memory, session recall, delegation, tool use, and scheduling. For that reason, Hermes closely mirrors the neuro-symbolic learning logic emphasized in NEOLAF, where prior knowledge guides action and experience updates knowledge in return.

Part I

Core Argument and Structural Correspondence

1 Framing the Problem

NEOLAF defines a learning encounter as a recursive loop among Knowledge, Situation, Task, Action, and Result. The importance of this formulation is that learning is not treated as passive accumulation of information, but as an ongoing cycle in which prior knowledge informs interpretation, interpretation shapes task selection, task selection drives action planning, action produces results, and results revise the knowledge state of the agent.

Within this loop, Action is not a minor intermediate step. It is the bridge between understanding and doing. If Action remains a vague natural-language suggestion rather than a manipulable system object, KSTAR stays at the level of conceptual description. To become operational, KSTAR needs Action to be implemented as a form of procedural knowledge that can be stored, reused, constrained, and revised. This is precisely why agent skill should be regarded as the best implementation mechanism for the action plan.

1.1 Why Action Cannot Remain at the Prompt Layer

NEOLAF does not define knowledge as a static inventory of facts. Instead, the paper characterizes knowledge in an action-oriented form that contains Situation, Task, Action plan, and Expected Result. This is a crucial design decision. Once Action is part of the internal structure of knowledge, it must be retrievable, inspectable, and updatable. A one-off prompt completion does not satisfy these requirements because it is difficult to stabilize, difficult to repair, and difficult to carry forward across encounters.

From an engineering perspective, prompts are useful for generating candidate plans, but they are poor containers for long-lived action knowledge. KSTAR therefore requires a more durable representational layer between model reasoning and real execution. A skill provides exactly that layer by transforming a fragile reasoning trace into a named, reusable capability that the agent can call again under comparable conditions.

2 Why Skill Is the Best Mechanism for the Action Plan

First, a skill gives Action a structured representation. NEOLAF treats Action as an ordered sequence of operations, not merely as an intention. The paper even notes that Action may be represented in a structured form such as an agent-skill-constraint triple. This is already very close to the software meaning of a skill: a named capability with explicit conditions, ordered steps, and operational boundaries.

Second, a skill unifies planning and execution. An action plan must do more than state what should happen; it must specify how the task is carried out, what tools are invoked, how intermediate steps are checked, what constraints apply, and how success is verified. Skills naturally package these elements together, turning Action into an executable blueprint rather than a disposable thought.

Third, a skill supports reuse and refinement. KSTAR is recursive, which means a successful or failed encounter should not vanish at the end of a turn. When the action plan is stored as a skill, newly discovered steps, missing constraints, common failure cases, and better verification routines can be written back into the same procedural object. This allows Action itself to become a target of learning.

2.1 The Boundary Between Skill and Memory

In an agent architecture, memory primarily stores facts, preferences, contextual details, and experience traces. It answers the question of what is known. A skill, by contrast, stores how a recurring class of tasks should be carried out. It answers the question of how to act. The Action component in KSTAR clearly belongs to the second category.

If Action is reduced to ordinary memory, the system loses precision: operational steps become mixed with broad context, and reuse becomes noisy. Separating skill from memory therefore matches the internal logic of KSTAR itself. Memory provides the contextual and experiential substrate, while skill provides the procedural mechanism that converts Task into Action.

2.2 How Skill Supports Feedback-Driven Revision

A defining feature of KSTAR is the comparison between the expected result and the actual result. When the two diverge, the system must revise its knowledge state in a way that is specific, interpretable, and repeatable. A skill gives the agent a concrete handle for such revision. If the failure arose from the wrong tool choice, a missing constraint, a poor step ordering, or an insufficient verification routine, the corresponding skill can be patched directly.

This makes the action plan an explicitly improvable object. A skill is therefore not just a reusable template; it is a procedural mechanism that accumulates corrective knowledge over time. This is one of the strongest reasons why skill is better suited to Action than a transient prompt output, which is far harder to update in a targeted and cumulative way.

3 Why Hermes Agent Resembles the NEOLAF Agent

The resemblance between Hermes Agent and the NEOLAF agent is not superficial. Both systems treat an agent not as a static language model that merely answers questions, but as a learning system capable of turning experience into explicit procedural knowledge. Hermes documentation explicitly describes a built-in learning loop in which the agent creates skills from experience, improves them during use, persists important information to memory, and recalls prior sessions through search. This closely parallels the NEOLAF cycle in which prior knowledge guides action and the resulting experience modifies future knowledge.

The similarity becomes even stronger when one looks at the role of skills. Hermes explicitly defines skills as procedural memory. This matches the role demanded by Action in NEOLAF. If NEOLAF requires Action to be a structured, executable, revisable knowledge object, then Hermes has already implemented precisely such an object under the name of skill.

3.1 Correspondence at the Neuro-Symbolic Level

NEOLAF emphasizes a neuro-symbolic division of labor: a large model interprets situations and proposes plans, while explicit symbolic mechanisms execute, constrain, verify, and store what has been learned. Hermes exhibits a very similar organization. The model side is responsible for understanding requests, selecting tools, and composing steps, whereas the system side provides explicit mechanisms such as skills, todo tracking, memory, delegation, session search, and scheduling.

This means Hermes derives its practical intelligence not from the model alone but from the interaction between model reasoning and structured external procedures. That is also the core insight of NEOLAF: a robust learning agent emerges when neural flexibility is coupled with explicit symbolic structures that preserve and regulate action knowledge.

3.2 Correspondence at the Level of Experience Consolidation

In NEOLAF, knowledge grows through repeated encounters in which experience is transformed into structured knowledge. Hermes provides concrete engineering support for the same pattern. Persistent memory stores facts and user models, session search recalls past episodes, skill management converts successful methods into reusable procedures, and delegation and scheduling extend the action plan across subtasks and time horizons.

For that reason, Hermes is not merely a tool-calling assistant. It already contains the machinery required to transform experience into reusable action knowledge and then feed that knowledge back into later decisions. In practical terms, it functions as a close software analogue of the learning logic formalized by NEOLAF, even if it does not replicate every theoretical component of the paper one by one.

4 Implications for KSTAR-Based Educational Agents

If KSTAR is to be implemented in educational systems, the action plan cannot remain a generic recommendation such as “practice more” or “review the concept.” Educational action unfolds over time: it requires prioritization, intervention design, verification of completion, diagnosis of failure, and later retesting. These demands make the action layer especially dependent on a stable procedural representation.

A skill is well suited to this role because it can preserve instructional or intervention logic while remaining open to revision when new evidence arrives. Once the action plan is realized as a skill, the agent can stop treating planning as ephemeral text generation and start treating it as a maintained capability. Only then does the KSTAR loop become a genuinely operational foundation for long-term adaptive learning systems.

5 Conclusion

Agent skill is the best implementation mechanism for the action plan in the KSTAR loop because it satisfies all of the properties that Action must have in a learning architecture: it is structured, executable, interpretable, reusable, and revisable. These properties allow Action to function not merely as a transient plan, but as a durable knowledge object that can improve over time through comparison between expected and actual results.

For the same reason, Hermes Agent closely resembles the NEOLAF agent at a structural level. Both systems move beyond the image of the agent as a chat model and toward the stronger idea of a learning system that converts experience into procedural knowledge and reuses that knowledge to guide subsequent action. From an implementation perspective, skill is therefore the most important bridge between the theoretical KSTAR loop and a working agent architecture.

References

- [1] Tong, R. J., Cao, C. C., Lee, T. X., Wan, R., Wang, F., and Hu, X. NEOLAF: A Neuro-Symbolic Cognitive Architecture for Experiential Learning. Preprint, 2025.
- [2] Tong, R. J., and Hu, X. Future of Education with Neuro-Symbolic AI Agents in Self-Improving Adaptive Instructional Systems. *Frontiers of Digital Education*, 1(2): 198–212, 2024.
- [3] Nous Research. Hermes Agent README, skills documentation, and skill manager implementation in the local repository snapshot accessed in April 2026.